ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Distributed Multi-Model Analytics for E-commerce Data

Individual Big Data Analytics Final Project

Student: Florence Imanishimwe

Student ID: 101399

Submission date: 11 November 2026

https://github.com/florishimwe17-sketch/AUCA-Big-Data-Analytics-Final-Project

Technical report | MongoDB, Apache HBase, Apache Spark, integrated analytics and visualization

Evidence basis: local execution outputs and screenshots supplied during project development. Figures reproduced from reported numeric results where the original image files were not available to this report generator.

1. Executive summary and objectives

This project demonstrates a distributed, multi-model analytics workflow for synthetic e-commerce data. Document-oriented records are analysed with MongoDB, user activity and daily product metrics are stored in Apache HBase, and Apache Spark is used for batch processing, SQL joins and an acquisition-to-purchase analysis. The goal is to combine storage models suited to different access patterns rather than force all data into one database.

The demonstrated sample includes 10,000 sessions. Spark identified 1,001 sessions linked to purchases, giving an actual purchase conversion rate of 10.01%. MongoDB category analysis ranked cat_019 (Powell PLC) first with $126,677.59 in line-item revenue; customer analysis ranked user_000104 first with $12,034.07 in total spending. HBase successfully stored and retrieved event records and daily metrics.

Project objectives

1. Generate structured but varied synthetic users, products, categories, transactions and web sessions.
2. Implement and query document and wide-column data models.
3. Execute Spark batch and SQL analytics across related datasets.
4. Analyse conversion and revenue, and communicate findings visually.

2. System architecture

Layer	Role and evidence
Synthetic JSON generator	Creates users, products, categories, transactions and sessions as JSON files.
MongoDB	Stores analytical collections and supports category revenue and customer spending aggregations.
Apache HBase 2.1.3 in Docker	Stores user_events and product_daily_metrics for keyed retrieval.
Apache Spark / PySpark	Reads JSON-derived DataFrames, performs joins and aggregations, and computes the purchase funnel.
Python / Matplotlib	Produces category revenue, session status and referral conversion charts.

The local deployment is a proof of concept: Spark runs in local mode and HBase runs in a single Docker container. These choices support functional validation but do not demonstrate multi-node throughput or fault tolerance.

3. Dataset and data modelling

The generated dataset includes five core JSON entities. Users contain customer attributes; products link to categories; categories provide category names; transactions contain user/session identifiers, timestamps and arrays of purchased items; and sessions contain page views, viewed products, cart contents, referral channels and conversion status. The generator was executed locally and produced files named users.json, products.json, categories.json, transactions.json and sessions_0.json.

MongoDB modelling

MongoDB is suited to transactions with nested line items and sessions with variable-length behavioural arrays. The project contains an import script (mongodb/import_data.py) and two analytical scripts (mongodb/category_revenue.py and mongodb/customer_analysis.py). Both analyses were run successfully against the local database. The category query groups completed transaction items by product category; the customer query ranks spending and order counts by user.

HBase row-key design

Table	Column families	Observed access pattern
user_events	event, session	User/event-oriented row keys; point lookup of an event record.
product_daily_metrics	metrics	Product/date-oriented row keys; scan or retrieve daily product view metrics.

The HBase shell confirmed that both tables were enabled. A sample event row used key user_test001#20261110#001 with event:type=product_view, event:product_id=prod_001 and session:session_id=session_test001. A get operation returned all three values. Subsequent counts showed 365 rows in user_events and 360 rows in product_daily_metrics. A sample product metrics scan returned metrics:views=1 for several product/date keys.

Data-quality considerations

The generated data is synthetic, so category names and country codes should not be interpreted as a real merchant portfolio or real customer geography. A session labelled converted does not necessarily have a linked transaction; the funnel analysis therefore distinguishes status labels from verified transaction linkage.

4. MongoDB analytics: verified results

4.1 Top product categories by revenue

Category	Revenue (USD)	Units	Transactions
cat_019 / Powell PLC	126,677.59	559	262
cat_013 / Smith Ltd	112,609.70	424	196
cat_007 / Moore, Becker and Carlson	103,383.65	330	155
cat_024 / Patterson, Smith and Jones	100,004.50	320	158
cat_018 / Phillips Inc	96,699.11	324	160

The leading category generated approximately $14,068 more revenue than the second-ranked category. Revenue is calculated from completed transaction line items using quantity multiplied by unit price. These are synthetic monetary values in the dataset, not audited business sales.

4.2 Highest-spending customers

Customer	Country code	Total spend (USD)	Orders
user_000104	VE	12,034.07	8
user_000367	MD	11,850.46	7
user_000496	RO	11,841.00	7
user_000210	WS	11,788.22	11
user_000127	KG	10,767.88	7

Customer user_000104 led total spending, with eight orders and an average reported order value of $1,504.26. Customer rankings provide a basis for segmentation, although no causal claims about loyalty or marketing effectiveness can be made from synthetic records.

Query validation

The two scripts completed without error in the supplied Command Prompt output and returned ranked results. The final repository should retain the scripts and document the database connection configuration and loading procedure for reproducibility.

5. Spark processing and integrated analytics

5.1 Spark SQL category analysis

Spark SQL joined transactions and nested items to products and categories to rank categories by gross line-item revenue, units sold and transaction counts. The top result was cat_019 (Powell PLC) with $126,677.59, 559 units and 262 transactions. The agreement with the MongoDB ranking provides a useful cross-check of the two analytical implementations.

5.2 Session-to-purchase funnel

Metric	Verified value
Sessions analysed	10,000
Sessions linked to a purchase	1,001
Actual purchase conversion rate	10.01%
Linked purchase revenue (USD)	791,391.30
Sessions labelled converted	2,009
Sessions labelled browsed	3,429
Sessions labelled abandoned	4,562

The integrated analysis links sessions with transactions to count observed purchasing sessions. Only 1,001 of the 2,009 sessions carrying a converted status were linked to an actual purchase in the reported analysis. Consequently, status-based conversion (20.09%) and transaction-linked purchase conversion (10.01%) must not be presented as interchangeable metrics.

5.3 Referral channel comparison

Channel	Sessions	Purchases	Purchase rate
Email	2,036	219	10.76%
Affiliate	2,013	208	10.33%
Social	1,952	195	9.99%
Direct	2,009	197	9.81%
Search engine	1,990	182	9.15%

Email had the highest observed purchase conversion rate, while search engine had the lowest. These descriptive differences do not establish statistically significant channel effects, and the synthetic data should not be used to recommend real advertising expenditure.

5.4 Product affinity and integration notes

Product affinity analysis (Apache Spark)

The updated spark/spark_analysis.py cleans transaction records, removes duplicate transaction IDs, flattens purchased items and counts distinct product pairs occurring in the same transaction. The pair counts are descriptive recommendation indicators, not a trained recommendation model.

Observed top product pairs

Product A	Product B	Co-purchases
prod_00178	prod_00187	4
prod_00037	prod_00214	3
prod_00099	prod_00184	3
prod_00064	prod_00182	3
prod_00145	prod_00165	3

Business interpretation

The leading pair, prod_00178 and prod_00187, appeared together in four transactions. Such co-purchases could inform cross-selling candidates, but the small observed frequencies do not establish predictive accuracy or causal relationships.

Cross-system architecture and limitations

MongoDB stores nested commerce records and supports document aggregations; HBase stores user events and daily product metrics using row-key-oriented access. Spark reads generated JSON-derived DataFrames and joins sessions with transactions, and transactions with product and category reference data. These are batch integrations of shared source data, not live MongoDB-to-HBase-to-Spark connector queries.

The proof-of-concept runs Spark in local mode and HBase in one Docker container. No cluster throughput, latency, or fault-tolerance benchmark was performed. For reproducibility, run the generator, database imports/loading, Spark scripts and visualization script in the documented order.

6. Visualizations and interpretation

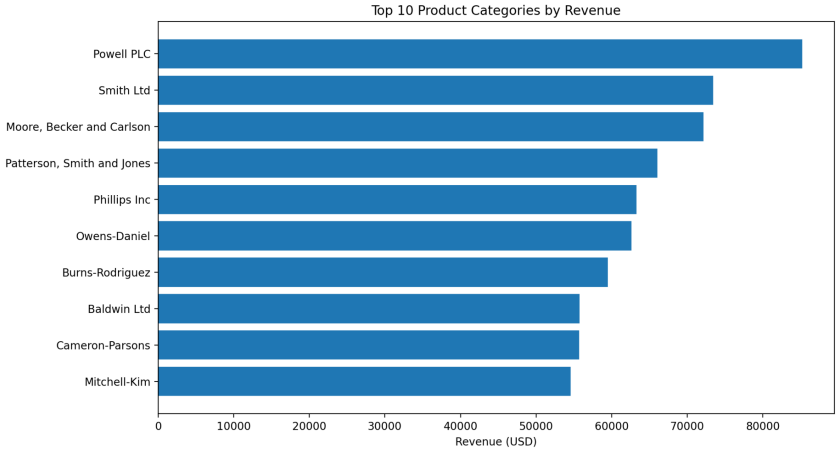


Figure 1. Top product categories by completed-transaction revenue

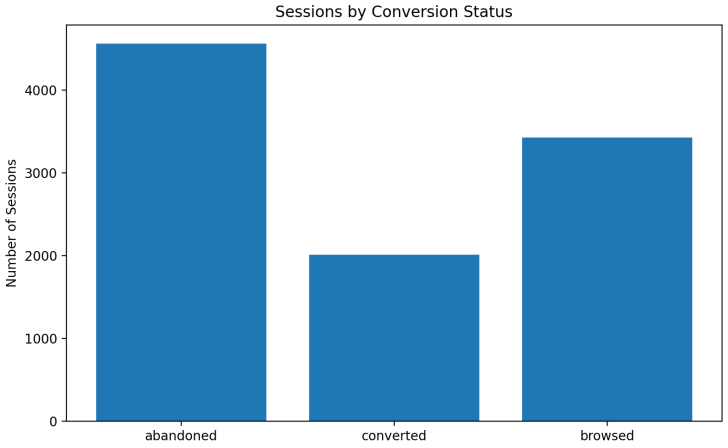


Figure 2. Session counts by reported conversion status

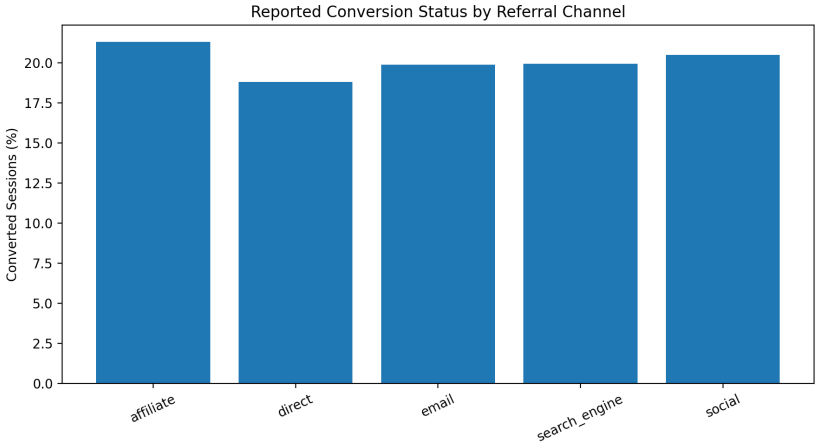


Figure 3. Reported conversion status by referral channel

Note: Figures 2-3 reflect session status labels, not transaction-linked purchases.

7. Scalability, limitations and conclusion

Scalability considerations

MongoDB can support indexing and aggregation over nested commerce documents; HBase supports sparse, wide-column records with row-key-oriented retrieval; and Spark distributes batch transformations across partitions when deployed on a cluster. A production implementation would require measured partitioning strategy, suitable HBase row-key distribution, database indexes, resource monitoring and a repeatable ingestion pipeline. The present tests validate functionality locally, not performance at production scale.

Limitations

First, the dataset is synthetic and its category names, referral behaviour and customer spending are generated rather than observed. Second, the Spark environment was tested in local mode and HBase was deployed in one container, so no cluster scalability benchmark was performed. Third, status-based session conversion and purchase-linked conversion diverge, illustrating the need for a precise business definition. Fourth, the report is based on console evidence provided during development; full reproducibility requires the final repository, environment details and original output files.

Conclusion

The project demonstrates a functioning multi-model analytics pipeline: MongoDB analyses successfully ranked category revenue and customer spending; HBase stored and returned user events and product/day metrics; Spark SQL computed category performance; and integrated Spark analysis measured 1,001 purchasing sessions from 10,000 sessions (10.01%). The three generated charts provide a visual summary of category revenue, session status and referral behaviour. The most important analytical lesson is to verify purchase events rather than assume a converted status implies a completed transaction.

Reproducibility and submission checklist

Before submission, verify the GitHub repository includes the JSON generator, MongoDB import and aggregation scripts, HBase schema/loading scripts, Spark processing/SQL/funnel scripts, visualization script, generated charts and README. The README should specify installation prerequisites, run order, database configuration and expected outputs. Capture evidence of HBase counts and retrieval, MongoDB aggregations, and Spark execution. Confirm the repository link is accessible to the examiner.

Implementation references

Project brief: AUCA Final Project Oct 2026.pdf (provided course assignment). Local source paths observed: data/dataset_generator_ready.py; mongodb/import_data.py, category_revenue.py, customer_analysis.py; visualizations/create_charts.py; Spark analysis scripts and HBase loading scripts within the corresponding project folders. Exact dependency versions and the final GitHub URL should be confirmed from the submitted repository.